

Assignment 2: Task Board

A Project Management Application

COP290 - Design Practices

Group Size: This assignment must be done in a **group of two**.

Deadline: To be announced.

Contents

1	Learning References	2
1.1	TypeScript	2
1.2	Style Guide	2
1.3	Frontend	2
1.4	Backend	2
1.5	Authentication	2
1.6	Database	3
1.7	Drag and Drop	3
2	Problem Statement: Task Board	4
2.1	Overview	4
2.2	Project Goals	4
2.3	Functional Requirements	4
2.3.1	Authentication, Authorization & User Management	4
2.3.2	Projects	5
2.3.3	Boards (Kanban)	5
2.3.4	Issues / Tasks	6
2.3.5	Issue Lifecycle & Workflow	6
2.3.6	Comments & Collaboration	7
2.3.7	Notifications	7
3	Evaluation Rubric	8
3.1	Code Quality Expectations	8
3.2	Testing Requirements	8
4	Submission Instructions	9

1 Learning References

This assignment must be implemented in **TypeScript**. Below are the recommended resources for learning and reference.

1.1 TypeScript

- **Official Documentation:** <https://www.typescriptlang.org/docs/>
- **TypeScript Handbook:** <https://www.typescriptlang.org/docs/handbook/intro.html>

1.2 Style Guide

Adhering to a consistent style guide is **mandatory** for this assignment:

- **Google TypeScript Style Guide:** <https://google.github.io/styleguide/tsguide.html>

Use the following tools to enforce style consistency:

- **ESLint:** <https://eslint.org/>
- **Prettier:** <https://prettier.io/>

1.3 Frontend

- **React with Vite:** <https://react.dev/>
 - Vite: <https://vitejs.dev/>
 - React with TypeScript: <https://react.dev/learn/typescript>
- **React Router:** <https://reactrouter.com/> (Client-side routing)
- **CSS Modules:** <https://github.com/css-modules/css-modules> (Scoped styling)
- **State Management:** Use React Context + useReducer (built-in)

1.4 Backend

- **Express.js:** <https://expressjs.com/>
- **REST API Design:** Design proper RESTful endpoints with appropriate HTTP methods and status codes

1.5 Authentication

- **JSON Web Tokens (JWT):** <https://jwt.io/introduction>
- **Password Hashing:** Passwords are securely hashed.(e.g., bcrypt).

1.6 Database

You may use any database of your choice. Some options:

- PostgreSQL: <https://www.postgresql.org/docs/>
- MySQL: <https://dev.mysql.com/doc/>
- MongoDB: <https://www.mongodb.com/docs/>
- SQLite: <https://www.sqlite.org/docs.html>

You may use an ORM (Object-Relational Mapping) library:

- Prisma: <https://www.prisma.io/docs> (Recommended for TypeScript)
- TypeORM: <https://typeorm.io/>
- Drizzle: <https://orm.drizzle.team/>

1.7 Drag and Drop

Implement drag-and-drop functionality using the native **HTML Drag and Drop API**. No external libraries allowed.

- MDN Drag and Drop API: https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API

2 Problem Statement: Task Board

Before starting this assignment, familiarize yourself with what Jira and Kanban boards are:

- **What is Jira?** <https://www.atlassian.com/software/jira/guides/getting-started/introduction>
- **What is a Kanban Board?** <https://www.atlassian.com/agile/kanban/boards>

2.1 Overview

A project management and issue-tracking application inspired by Jira. It enables teams to plan, track, and manage work using boards, issues, workflows, and more.

2.2 Project Goals

- Provide a Kanban-style task tracking system ✓
- Support projects, boards, issues, and workflows ✓
- Enable team collaboration and role-based access ✓
- Maintain auditability and traceability of changes

2.3 Functional Requirements

2.3.1 Authentication, Authorization & User Management

User Registration and Login

- Email + password authentication ✓
- Passwords must be hashed ✓

Session Management

- Use JWT (JSON Web Tokens) for authentication ✓
- Access tokens stored in HTTP-only cookies ✓
- Implement refresh token mechanism for session renewal ✓
- Logout must invalidate the refresh token ✓

User Profile

- Name ✓
- Email ✓
- Avatar upload and display ✓

Role-Based Access Control

There are two levels of roles:

Global Roles:

- **Global Admin:** Can create projects, manage all users, and assign project-level roles ✓

Project-Level Roles (users can have different roles across different projects):

- **Project Admin:** Full access to project settings, can manage project members and edit workflows ✓

- **Project Member:** Create and edit tasks, participate in projects ✓

- **Project Viewer:** Read-only access to project and its tasks ✓

Only Global Admins and Project Admins can assign roles within a project.

2.3.2 Projects

Project Management

- Create, update, and archive projects ✓
- Assign users to projects ✓
- Project-level permissions ✓

Project Metadata

- Name
- Description
- Creation and modification timestamps

2.3.3 Boards (Kanban)

Board Structure

- Each project has one or more boards ✓
- Each board has its own workflow (column configuration) ✓
- Each column corresponds to a workflow status ✓
- Default columns: To Do, In Progress, Review, Done (customizable per board) ✓
- Drag-and-drop tasks between columns
- Column-level WIP (Work In Progress) limits must be enforced — invalid moves should be blocked, not just warned

Board Features

- ★ • Add, rename, reorder, and delete columns
- Visual feedback during drag operations ✓
- Only Project Admins can modify board workflows (add/rename/delete columns) ✓

2.3.4 Issues / Tasks

Task Types (Hierarchical)

- **Story:** Parent item that contains related Tasks and Bugs (not directly movable across columns) ✓
- **Task:** Standard work item, can be linked to a Story as a child ✓
- **Bug:** Issue to be fixed, can be linked to a Story as a child ✓

Parent-child relationships must be enforced. A Story's status can be automatically derived from its children's statuses, or manually set. If manually set, the Story status must be consistent with its children's states.

Task Fields

- Title
- Description
- Status (based on workflow)
- Priority (Low, Medium, High, Critical)
- Assignee (team member assigned to task)
- Reporter (user who created the task)
- Due date

Task Operations

- Create, edit, and delete tasks
- Move tasks between columns
- Change task status, priority, and assignment
- Task details view with full information

2.3.5 Issue Lifecycle & Workflow

- Configurable workflows per board
- Valid status transitions (e.g., To Do → In Progress → Done)
- Invalid status transitions must be blocked

Audit Trail (Mandatory)

The following events must be logged for each task:

- Status changes (with old and new status)
- Assignee changes (with old and new assignee)
- Comments added/edited/deleted

- ✓ • All timestamps for above events

Note: Viewing audit history UI is not required; storing the audit data is sufficient.

Automatic Timestamps

- Created timestamp
- Updated timestamp
- Resolved timestamp
- ✓ • Closed timestamp

2.3.6 Comments & Collaboration

- ✓ • Comment threads on tasks
- ✓ • Edit and delete own comments
- ✓ • Mention users using @username notation
- ✓ • Activity timeline per task showing all changes and comments

- ~~✗~~ • Rich text support in comments

2.3.7 Notifications

Notification Triggers

- ✓ • Task assigned to user
- ✓ • Task status changed
- ~~✗~~ • Comment added to task
- ✓ • User mentioned in comment

Delivery Methods

- ~~✗~~ • In-app notifications with notification center
- ✓ • Notifications must be persistent (stored in database)
- ✓ • Users can mark notifications as read
- ✓ • Notification history must be viewable

Polling-based notification fetching is acceptable; real-time (WebSocket) delivery is optional.

3 Evaluation Rubric

Your submission will be evaluated based on the following criteria:

Component	Weightage	Description
Functionality	40%	All required features working correctly, proper error handling, edge cases covered
Code Quality, Style & Testing	25%	Adherence to TypeScript style guide, clean code, proper typing, no <code>any</code> types, consistent formatting, backend unit tests
UI/UX Design	15%	Intuitive interface, responsive design, visual feedback, accessibility considerations
Database Design	10%	Proper schema design, relationships, indexing, data integrity
Documentation	10%	README with setup instructions, API documentation, code comments where necessary

3.1 Code Quality Expectations

- Use strict TypeScript configuration (`strict: true` in `tsconfig.json`)
- No usage of `any` type unless absolutely necessary (and justified)
- Proper error handling with typed errors
- Consistent naming conventions
- Modular and reusable code structure

3.2 Testing Requirements

- **Backend unit tests are mandatory** and will be graded
- Test critical API endpoints and business logic
- Frontend testing is optional but encouraged

4 Submission Instructions

1. You need to submit your source code, README (with compile/run instructions) and `report.pdf`.
2. **DO NOT** use any libraries other than those mentioned in this document.
3. We will use modern AI-based plagiarism checkers. If there is a match, you will receive **zero** in this assignment and another assignment of our choosing.
4. There may additionally be a **demo and viva**. It is possible that the assignment works perfectly but the viva performance falls short of expectations. In this case, marks may be significantly reduced. The demo/viva performance is thus very important.
5. Write a report indicating your design decisions and include your GitHub repository link in the report. **Both team members must commit code from their own GitHub accounts — no exceptions.**
6. All submissions will be via **Moodle**. Create a tar file as follows:
 - (a) Assume all your code is in a directory named `assignment2`.
 - (b) Go to the parent directory of the project.
 - (c) Run: `tar -cvf a2_<entry_number1>_<entry_number2>.tar assignment2`
 - (d) Submit a file named `a2_<entry_number1>_<entry_number2>.tar`
Example: If your entry numbers are 2021CS98765 and 2021CS98763, submit:
`a2_2021CS98765_2021CS98763.tar`
7. Submit several hours before the deadline. Double-check your submission.
8. **No emails will be entertained.**